SUNDAY, MAY 24, 2026 Browser Edition — No Server Required VOL. I

QRCodex

INSTANT QR CODES
GENERATED LOCALLY

GENERATOR HOW IT WORKS ABOUT QR

— MINI PROJECT REPORT

QR CODEX: INSTANT QR CODE GENERATOR

A client-side QR code generation tool built on the ISO/IEC 18004 standard with Reed-Solomon error correction, configurable output sizing, and local-only processing.

NO SERVER · NO TRACKING · NO UPLOAD

HTML5 CSS3 JavaScript Netlify

SUBMITTED BY

SAMANESH

ROLL NUMBER
24BCA107

DEPARTMENT
BCA

SEMESTER
4th Semester

SUBMISSION
July 2026

ST. CLARET COLLEGE (AUTONOMOUS)
Jalahalli, Bengaluru – 560057

qr-codex-g.netlify.app

Project Report submitted in partial fulfillment of the requirements for the degree of Bachelor of Computer Applications (BCA)

CERTIFICATE

This is to certify that the mini project entitled

QR CODEX — INSTANT QR CODE GENERATOR

has been successfully developed and submitted by **SAMANESH** (Roll No. **24BCA107**), a student of the 4th Semester, Bachelor of Computer Applications (BCA), St. Claret College (Autonomous), Bengaluru, in partial fulfillment of the academic requirements for the degree of Bachelor of Computer Applications.

This mini project was developed independently as part of self-learning and skill enhancement during the vacation period. The work presented in this report is original and has not been submitted elsewhere for any academic examination or award.

The project demonstrates a thorough understanding of QR code generation algorithms, browser-based computation, and the principles of privacy-first software design.

St. Claret College (Autonomous), Bengaluru

DECLARATION

I, **SAMANESH**, student of 4th Semester, Bachelor of Computer Applications (BCA), St. Claret College (Autonomous), Bengaluru (Roll No. 24BCA107), hereby declare that the mini project report entitled **"QR CODEX — INSTANT QR CODE GENERATOR"** submitted to the Department of BCA is a record of original work done by me during the vacation period.

I further declare that:

- This project has been carried out independently, without any formal institutional guidance, as a self-initiated learning exercise.
- The work presented in this report is genuine and has not been copied or reproduced from any other source without proper acknowledgement.
- This project has not been submitted previously for any degree, diploma, or academic title at this or any other institution.
- All references and sources of information used have been duly cited and acknowledged.
- The analysis performed and the conclusions drawn are based entirely on my own understanding and research.

Place: Bengaluru

Date: July 2026

Signature of the Student

SAMANESH

Roll No: 24BCA107

4th Semester, BCA

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who encouraged and supported me throughout the development of this mini project.

I am grateful to the open-source developer community and the educators whose resources — including MDN Web Docs, the ISO/IEC QR Code specification documentation, Stack Overflow discussions, and YouTube tutorials — guided my understanding of QR code algorithms, Reed-Solomon error correction, and JavaScript canvas rendering techniques.

I sincerely thank my family for their constant encouragement and for providing a focused environment for learning and development during the vacation period.

Implementing a QR code generator from algorithmic first principles — without relying on a third-party QR library — was a deeply challenging and rewarding exercise. It sharpened my understanding of binary encoding, matrix construction, and error correction mathematics in ways that no classroom exercise could have replicated.

SAMANESH

Roll No: 24BCA107

4th Semester, BCA

St. Claret College (Autonomous), Bengaluru

ABSTRACT

QR (Quick Response) codes have become a ubiquitous part of modern digital life, used for everything from payment systems and product labelling to contact sharing and website linking. Despite their widespread use, most individuals rely on online QR generation services that transmit user data to remote servers — raising significant privacy concerns, particularly when the content being encoded is sensitive.

QR Codex is a browser-based QR code generator that implements the QR Code specification (ISO/IEC 18004) entirely in JavaScript, within the user's browser. No data is ever transmitted to any server. The application provides a clean, newspaper-inspired interface for entering text or URLs, configuring output parameters, and downloading the generated QR code — all with zero network dependency after the initial page load.

Key Features:

- Data/URL input field supporting up to 2,953 characters with a live character counter.
- Output size slider for pixel-perfect control of the generated QR code dimensions.
- Four error correction levels: L (Low, ~7% recovery), M (Medium, ~15%), Q (Quartile, ~25%), H (High, ~30%).
- Real-time QR code rendering in the output panel with metadata display (dimensions, ECL, version, character count).
- One-click Download of the QR code as a PNG image.
- One-click Copy to clipboard for immediate use.
- Expandable 'How It Works' section explaining the encoding algorithm transparently.
- Fully offline-capable after first load — no server, no tracking, no data upload.

Technologies Used:

HTML5, CSS3, Vanilla JavaScript (ISO/IEC 18004 QR algorithm implementation), HTML5 Canvas API. Deployed on Netlify.

Expected Outcome:

QR Codex provides a private, instant, and configurable QR code generation experience that eliminates the need for external services, demonstrating that complex encoding standards can be implemented effectively in the browser using only the core web platform.

TABLE OF CONTENTS

Certificate	2
Declaration	3
Acknowledgement	4
Abstract	5
Table of Contents	6
Chapter 1 – Introduction	7
Chapter 2 – Objectives of the Project	8
Chapter 3 – Existing System	9
Chapter 4 – Proposed System	10
Chapter 5 – Modules and Features	11
Chapter 6 – Technologies Used	13
Chapter 7 – Conclusion	15

CHAPTER 1

INTRODUCTION

1.1 Background

QR codes — Quick Response codes — were developed in 1994 by Denso Wave and have since become one of the most widely adopted two-dimensional barcode formats in the world. Their ability to encode a wide variety of data types (URLs, plain text, contact information, payment details) in a compact, machine-readable format has made them indispensable across industries including retail, logistics, healthcare, marketing, and digital payments.

Despite the ubiquity of QR codes, most individuals who need to generate one rely on online tools that process the input on remote servers. This approach is convenient but raises serious privacy concerns: the content being encoded — which may include personal URLs, contact details, or sensitive information — is transmitted to and potentially logged by a third-party server. A locally-executed alternative eliminates this concern entirely.

1.2 About the Project

QR Codex (Browser Edition, Vol. I) is a single-page web application that generates QR codes entirely within the user's browser using a pure JavaScript implementation of the QR Code standard (ISO/IEC 18004). The application features a distinctive newspaper-broadsheet-inspired aesthetic, offering an interface that is both visually memorable and functionally streamlined.

The application is live at: **https://qr-codex-g.netlify.app/**, deployed via Netlify. It requires no installation, no login, and no internet connection beyond the initial page load.

1.3 Scope

- Client-side QR code generation implementing the ISO/IEC 18004 standard.
- Support for text and URL encoding up to 2,953 characters.
- Configurable output size and four Reed-Solomon error correction levels.
- Real-time rendering with QR version and metadata display.
- PNG download and clipboard copy functionality.
- Educational transparency through an in-page algorithmic explanation.
- Complete data privacy — zero server communication.

CHAPTER 2

OBJECTIVES OF THE PROJECT

The following objectives guided the design and development of QR Codex:

1. To implement the QR Code standard in the browser: Develop a JavaScript implementation of the ISO/IEC 18004 QR Code specification, including data encoding, Reed-Solomon error correction, matrix construction, and masking, without relying on any third-party QR library.

2. To ensure absolute user data privacy: Design the system so that input data never leaves the user's device; all computation must occur client-side, with no server requests initiated during or after QR generation.

3. To provide configurable error correction: Offer four error correction levels (L, M, Q, H) corresponding to different data recovery capacities, allowing users to select the appropriate level based on their use case.

4. To deliver flexible output options: Allow users to control the pixel dimensions of the generated QR code via a slider, and provide both download (PNG) and clipboard copy functionality for ease of use.

5. To educate users about QR technology: Include a transparent, in-page explanation of the QR generation algorithm — covering binary encoding, Reed-Solomon mathematics, and mask pattern selection — to demystify the technology for curious users.

6. To create a distinctive, memorable user interface: Design the application with a newspaper-broadsheet aesthetic that sets it apart visually from generic tool interfaces while maintaining full usability and readability.

7. To demonstrate advanced JavaScript programming competence: Apply complex algorithmic concepts including Galois field arithmetic, polynomial division, and matrix operations in a functional, production-deployed web application.

CHAPTER 3

EXISTING SYSTEM

3.1 Current QR Code Generation Methods

Users currently rely on several approaches to generate QR codes, each with notable drawbacks:

a) Server-Side Online QR Generators

The majority of QR generation tools (QR Code Monkey, GoQR.me, QR Tiger, etc.) process input data on their servers before returning the generated image. This means the user's data — including potentially sensitive URLs, contact details, or payment information — is transmitted to and may be logged by a third-party server. Many such services also require account creation for advanced features.

b) Mobile QR Generator Apps

Smartphone applications for QR generation frequently include advertisements, require permissions, and may upload data for analytics. They are device-specific and offer limited configuration compared to a well-designed web tool.

c) Desktop QR Software

Standalone desktop applications offer offline processing but require installation, are platform-specific, and are typically not free for commercial use. They are impractical for quick, on-demand generation in a browser environment.

d) QR Libraries in Code

Developers can use QR libraries (qrcode.js, python-qrcode) programmatically, but this requires technical knowledge and is not accessible to non-developer users who simply need to generate a QR code quickly.

3.2 Limitations of Existing Solutions

- Server-side processing compromises input data privacy.
- Most free tools include usage limitations, watermarks, or advertisements.
- Account registration barriers for advanced features.
- No transparency about the underlying encoding algorithm.
- Limited error correction level selection in consumer-facing tools.
- Mobile apps require installation and may request unnecessary permissions.
- No single tool combines privacy, configurability, algorithm education, and a refined UI.

CHAPTER 4

PROPOSED SYSTEM

4.1 Overview

QR Codex proposes a browser-native QR code generator that processes all data locally using a pure JavaScript implementation of the ISO/IEC 18004 standard. It combines absolute privacy, configurable generation parameters, multiple output options, and algorithmic transparency in a single, installation-free web application with a distinctive editorial aesthetic.

4.2 Advantages Over Existing Systems

- **Absolute Privacy:** Input data is never transmitted to any server. All computation occurs in the browser using the user's own hardware.
- **No Account or Installation Required:** Open the URL and generate a QR code immediately with zero barriers.
- **Configurable Error Correction:** Four levels (L/M/Q/H) give users control over the trade-off between data capacity and damage resilience.
- **Flexible Output:** Pixel-size slider, PNG download, and clipboard copy serve different use cases from print to digital.
- **Algorithmic Transparency:** The in-page 'How It Works' section explains the encoding process, differentiating QR Codex from black-box generator tools.
- **Distinctive Design:** The newspaper-broadsheet UI is immediately recognisable and memorable, reflecting a level of design thinking uncommon in utility tools.
- **Standards Compliance:** The ISO/IEC 18004 implementation ensures generated codes are compatible with all standard QR scanners.

4.3 System Architecture

QR Codex is implemented as a single HTML file with embedded CSS and JavaScript. The application has no external runtime dependencies for its core QR generation logic — the ISO/IEC 18004 algorithm is implemented entirely in Vanilla JavaScript, using the HTML5 Canvas API for rendering. The three navigation sections (Generator, How It Works, About QR) exist within the same page, with smooth scroll or show/hide toggling. The application is served as a static file from Netlify's CDN.

CHAPTER 5

MODULES AND FEATURES

QR Codex is structured around three primary sections accessible via the top navigation bar: Generator, How It Works, and About QR.

5.1 Generator Section (COMPOSE Panel)

The Generator is the core functional section of the application, divided into a COMPOSE panel on the left and an OUTPUT panel on the right.

Data / URL Input:

A multi-line text area accepts any text or URL for encoding, with a live character counter displaying the current count against the maximum capacity of 2,953 characters (the maximum for QR Version 40 at error correction level L). An asterisk marks it as a required field.

Output Size Slider:

A range slider controls the pixel dimensions of the generated QR code, with the current value displayed in real time (default: 200 px, as seen in the screenshot showing 200 px selected). This allows users to generate codes optimised for both screen display and high-resolution print.

Error Correction Level Selector:

Four toggle buttons — L, M, Q, H — allow selection of the Reed-Solomon error correction level. Each level determines what percentage of the QR code's data can be reconstructed if the code is damaged or partially obscured:

Level	Name	Recovery Capacity	Typical Use Case
L	Low	~7%	Clean digital environments (screens)
M	Medium	~15%	General purpose (default)
Q	Quartile	~25%	Industrial or print applications
H	High	~30%	Logos embedded in QR code

Action Buttons:

- ■ **GENERATE:** Triggers the QR encoding algorithm with the current configuration, rendering the result in the OUTPUT panel.
- ↓ **DOWNLOAD:** Saves the generated QR code as a PNG image file to the user's device.
- ■ **COPY:** Copies the QR code image to the system clipboard for immediate pasting.

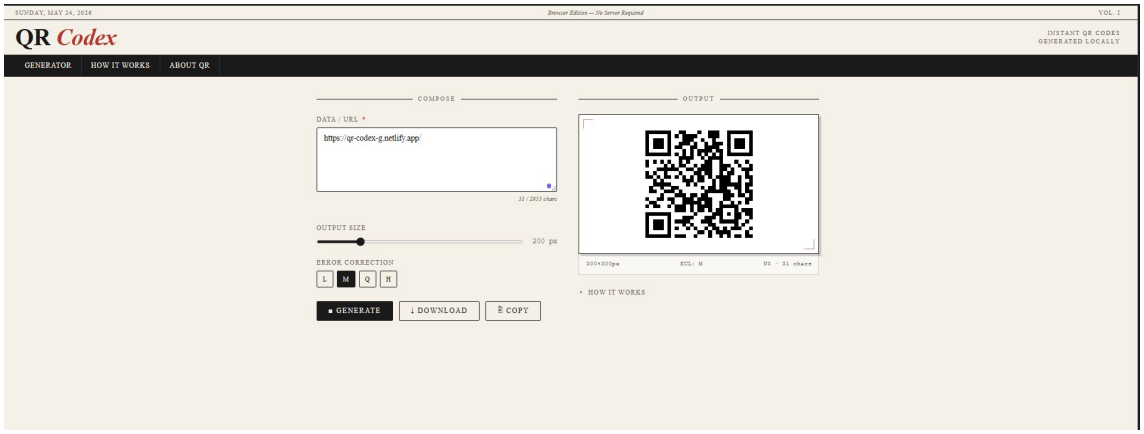


Figure 5.1 – QR Codex Generator Interface with COMPOSE and OUTPUT Panels

5.2 OUTPUT Panel

The right-hand output panel renders the generated QR code in real time using the HTML5 Canvas API. Below the QR image, a metadata bar displays:

- **Dimensions:** The pixel size of the output (e.g., 200×200px).
- **ECL:** The active error correction level (e.g., ECL: M).
- **Version:** The QR version number, which determines the module grid size (e.g., V3).
- **Character Count:** The number of characters encoded (e.g., 31 chars).

5.3 How It Works Section

An expandable '■ How It Works' panel below the output area explains the QR generation process in plain language for educational purposes. It describes the binary matrix encoding process, the application of Reed-Solomon error correction at the chosen level, and the mask pattern selection optimised for scanner readability. This commitment to algorithmic transparency is a defining characteristic of QR Codex.

5.4 About QR Section

A navigation link points to the Wikipedia article on QR codes, providing users with authoritative reference material about the standard that underpins the application. The application footer reaffirms its privacy-first design: *"All processing local — No tracking — No server."*

CHAPTER 6

TECHNOLOGIES USED

QR Codex was developed using only the native web platform. No external QR library or JavaScript framework was used.

6.1 HTML5

HTML5 forms the structural backbone of QR Codex. The single-file application uses semantic HTML5 elements for the navigation bar, compose and output panels, form controls (textarea, range slider, radio buttons), and the canvas element for QR rendering. The File API enables the PNG download functionality.

6.2 CSS3

CSS3 delivers QR Codex's distinctive newspaper-broadsheet aesthetic. The design uses a warm parchment-cream background (#f5f0e8), near-black text, and crimson-red accents to evoke a printed editorial feel. A monospace font in the input area reinforces the technical character of the tool. CSS Grid handles the two-column compose/output layout, while CSS transitions animate the error correction button states.

6.3 JavaScript (ES6+, Vanilla) — ISO/IEC 18004 Implementation

The most technically significant component of the project is the pure JavaScript implementation of the QR Code standard. Key algorithmic steps implemented include: data mode selection and binary encoding, Galois field (GF(256)) arithmetic for Reed-Solomon polynomial computation, QR version selection based on data length and error correction level, matrix construction including finder patterns, timing patterns, and alignment patterns, mask pattern evaluation and selection using the QR penalty scoring rules, and format and version information encoding.

6.4 HTML5 Canvas API

The Canvas API is used to render the QR code as a pixel matrix. Each module (black or white cell) of the QR code grid is drawn as a filled rectangle on the canvas. The canvas is also the source for the PNG download and clipboard copy operations, via the toDataURL() and toBlob() methods respectively.

6.5 Netlify

The application is deployed as a static site on Netlify, providing HTTPS, global CDN distribution, and a custom subdomain. The entirely static nature of the project (single HTML file, no build step required) makes Netlify an ideal hosting choice.

#	Technology	Type	Purpose

1	HTML5	Markup	Structure, canvas element, file API
2	CSS3	Styling	Broadsheet theme, layout, button states
3	JavaScript ES6+	Programming	ISO/IEC 18004 QR algorithm, DOM, events
4	Canvas API	Browser API	QR matrix rendering, PNG export, clipboard
5	Netlify	Platform	Static hosting, HTTPS, CDN

CHAPTER 7

CONCLUSION

QR Codex is a technically ambitious and well-executed mini project that sets itself apart from typical student web applications through the depth of its algorithmic implementation. Developing a standards-compliant QR code generator from first principles — including Galois field arithmetic, Reed-Solomon polynomial computation, QR matrix construction, and mask pattern scoring — without relying on any external QR library represents a significant programming achievement for a student at the 4th Semester BCA level.

The application's commitment to privacy — processing all data locally with zero server communication — reflects a principled approach to software design that is increasingly valued in the professional software industry. The decision to include a transparent algorithmic explanation within the application itself demonstrates an educational intent that goes beyond mere tool development.

From a design perspective, the newspaper-broadsheet aesthetic is distinctive, intentional, and consistently executed — from the dark masthead bar and crimson typography to the parchment background and editorial type treatments. This level of visual design thinking reflects an understanding that user experience extends beyond functionality.

On a personal level, undertaking this project independently during the vacation period has been an exceptionally enriching experience. The process of studying the ISO/IEC 18004 standard, implementing its algorithms in JavaScript, debugging complex binary matrix constructions, and deploying a polished, functional application has substantially deepened both technical knowledge and the confidence to tackle complex programming challenges.

In conclusion, QR Codex is a polished, standards-compliant, privacy-first QR code generator that fully realises its stated objectives and stands as a strong demonstration of advanced front-end development skill, algorithmic understanding, and thoughtful product design.

Live Project: https://qr-codex-g.netlify.app/